

Chat Application Project Report

Abstract

This report describes a simple chat application built for users to communicate in real-time. The app allows users to register, log in, choose a display name, and chat in different rooms. It supports text messages and file uploads like images and audio. The project uses modern web technologies to create a user-friendly and secure chat experience.

1 Introduction

The chat application is a web-based platform where users can connect and chat in real-time. It was created to provide a simple way for people to talk in group rooms or send private messages. The app ensures user security with features like password protection and safe input handling. This report explains the tools used, steps to build the project, and key outcomes.

2 Tools Used

The project was built using the following tools and technologies:

- **Python:** Used for the backend with the Flask framework to handle web requests and manage the app.
- **HTML/CSS:** Created the user interface for login, registration, nickname selection, and chat pages.
- **JavaScript:** Powered real-time chat features using Socket.IO for instant messaging.
- **SQLAlchemy:** Managed the database to store user accounts securely.
- **Bleach:** Ensured safe user input to prevent harmful code.
- **Flask-Limiter:** Limited requests to protect against abuse.
- **Flask-WTF:** Added CSRF protection for secure forms.
- **SQLite:** Stored user data in a lightweight database.

3 Steps Involved in Building the Project

The project was developed in the following steps:

1. **Planning:** Designed the app structure with login, registration, nickname selection, and chat interfaces. Defined chat rooms and file upload features.
2. **Backend Setup:** Used Flask to create routes for user registration, login, and nickname selection. Added SQLite database with SQLAlchemy for user management.
3. **Frontend Development:** Built HTML pages for login, registration, nickname selection, and chat. Added CSS for a clean, modern look.
4. **Real-Time Chat:** Integrated Socket.IO to handle real-time messaging, room joining, and user status updates.
5. **Security:** Added password hashing, CSRF protection, and input sanitization to keep the app safe.

6. **File Uploads:** Enabled secure file uploads for images and audio, with limits on file types and sizes.
7. **Testing:** Tested the app to ensure registration, login, chat, and file uploads worked smoothly.

4 Conclusion

The chat application was successfully built to provide a simple and secure way for users to communicate. It supports real-time messaging, multiple chat rooms, and file sharing. The project uses reliable tools like Flask, Socket.IO, and SQLite to ensure performance and security. Future improvements could include adding video chat or mobile app support. This project shows how web technologies can create an effective communication platform.